

Cahier des charges

Projet intégrateur : Assistant intelligent semi-autonome d'investigation numérique
Version 2.0 – Spécifications exhaustives

Responsable pédagogique : Prof. Thierry MINKA
Classe : 4^e année Ingénieur en cybersécurité (27 étudiants)

24 octobre 2025

Table des matières

1	Résumé exécutif	6
1.1	Objectif du projet	6
1.2	Durée et organisation	6
1.3	Livrables attendus	6
1.4	Indicateurs de succès	6
2	Contexte et justification	6
2.1	Contexte pédagogique	6
2.2	Contexte professionnel	7
2.3	Enjeux techniques	7
3	Objectifs	7
3.1	Objectifs généraux	7
3.2	Objectifs pédagogiques	7
3.3	Objectifs techniques	7
4	Périmètre fonctionnel	7
4.1	MVP (Minimum Viable Product) – Obligatoire	7
4.2	Extensions (Stretch Goals)	8
4.3	Hors périmètre	8
5	Architecture fonctionnelle globale	8
5.1	Vue d'ensemble	8
5.2	Flux principaux	8
5.2.1	Phase 1 : Création du mandat	8
5.2.2	Phase 2 : Préparation de la mission	9
5.2.3	Phase 3 : Analyse des preuves	9
5.2.4	Phase 4 : Rapport final	9
5.3	Schéma d'architecture détaillé	9
5.4	Principes architecturaux	9
6	Stack technologique imposée	10
6.1	Backend	10
6.2	Frontend	10
6.3	NLP et Machine Learning	10
6.4	Génération de documents	10
6.5	Sécurité et journaux	10

6.6	DevOps et infrastructure	10
6.7	Outils de développement	10
6.8	Justification des choix	11
7	Fiches modules détaillées	11
7.1	Module A : Interface Mandataire (Front-end Mandataire)	11
7.1.1	Fonctions principales (MVP)	11
7.1.2	Extensions	11
7.1.3	Entrées/Sorties	11
7.1.4	API exposée	11
7.1.5	Dépendances	12
7.1.6	Livrables	12
7.1.7	Critères d'acceptation	12
7.1.8	Métriques de qualité	12
7.2	Module B : Expert Matching (Profil, Scoring, Base CV)	12
7.2.1	Fonctions principales (MVP)	12
7.2.2	Extensions	12
7.2.3	Modèle de données expert	13
7.2.4	Algorithme de scoring détaillé	13
7.2.5	API exposée	13
7.2.6	Dépendances	14
7.2.7	Livrables	14
7.2.8	Critères d'acceptation	14
7.2.9	Métriques de qualité	14
7.3	Module C : Générateur de Mandat / Contrats	14
7.3.1	Fonctions principales (MVP)	15
7.3.2	Extensions	15
7.3.3	Template LaTeX (extrait)	15
7.3.4	Processus de génération	15
7.3.5	API exposée	16
7.3.6	Dépendances	16
7.3.7	Livrables	16
7.3.8	Critères d'acceptation	17
7.3.9	Métriques de qualité	17
7.4	Module D : Workflow d'Acceptation	17
7.4.1	Fonctions principales (MVP)	17
7.4.2	Extensions	17
7.4.3	Modèle de données	17
7.4.4	Machine à états	17
7.4.5	API exposée	18
7.4.6	Dépendances	18
7.4.7	Livrables	18
7.4.8	Critères d'acceptation	19
7.4.9	Métriques de qualité	19
7.5	Module E : Préparation Mission	19
7.5.1	Fonctions principales (MVP)	19
7.5.2	Extensions	19
7.5.3	Génération de questions (règles)	19
7.5.4	API exposée	20
7.5.5	Dépendances	20
7.5.6	Livrables	20
7.5.7	Critères d'acceptation	20

7.5.8	Métriques de qualité	21
7.6	Module F : Ingestion et Analyse PV	21
7.6.1	Fonctions principales (MVP)	21
7.6.2	Extensions	21
7.6.3	Pipeline NLP détaillé	21
7.6.4	Spécification des incohérences	22
7.6.5	API exposée	22
7.6.6	Dépendances	23
7.6.7	Livrables	23
7.6.8	Critères d'acceptation	23
7.6.9	Métriques de qualité	23
7.7	Module G : Hypothèses et Méthodologies Techniques	23
7.7.1	Fonctions principales (MVP)	23
7.7.2	Extensions	23
7.7.3	Base de connaissances forensiques	24
7.7.4	Génération du plan méthodologique	24
7.7.5	API exposée	25
7.7.6	Dépendances	25
7.7.7	Livrables	25
7.7.8	Critères d'acceptation	25
7.7.9	Métriques de qualité	26
7.8	Module H : Analyse Rapports Techniques	26
7.8.1	Fonctions principales (MVP)	26
7.8.2	Extensions	26
7.8.3	Critères de scoring	26
7.8.4	Chaîne optimale	26
7.8.5	API exposée	27
7.8.6	Dépendances	28
7.8.7	Livrables	28
7.8.8	Critères d'acceptation	28
7.8.9	Métriques de qualité	28
7.9	Module I : Générateur Rapport Expertise	28
7.9.1	Fonctions principales (MVP)	28
7.9.2	Extensions	28
7.9.3	Structure du rapport	29
7.9.4	Template LaTeX (extrait)	29
7.9.5	API exposée	29
7.9.6	Dépendances	30
7.9.7	Livrables	30
7.9.8	Critères d'acceptation	30
7.9.9	Métriques de qualité	30
7.10	Module J : Sécurité et Journaux Immuables	30
7.10.1	Fonctions principales (MVP)	30
7.10.2	Extensions	30
7.10.3	Format du journal	30
7.10.4	Chaîne de custody	31
7.10.5	API exposée	31
7.10.6	Dépendances	32
7.10.7	Livrables	32
7.10.8	Critères d'acceptation	32
7.10.9	Métriques de qualité	32

7.11	Module K : Explicabilité (XAI)	32
7.11.1	Fonctions principales (MVP)	32
7.11.2	Extensions	33
7.11.3	Types d'explications	33
7.11.4	API exposée	33
7.11.5	Dépendances	34
7.11.6	Livrables	34
7.11.7	Critères d'acceptation	34
7.11.8	Métriques de qualité	34
8	Tableau des interactions inter-modules	35
9	Contraintes non-fonctionnelles	35
9.1	Performance	35
9.2	Sécurité	35
9.3	Forensic et légal	36
9.4	Scalabilité	36
9.5	Disponibilité	36
9.6	Maintenabilité	36
10	Organisation et gouvernance	36
10.1	Répartition des équipes	36
10.2	Équipe d'intégration	37
10.3	Méthodologie Agile	37
10.4	Livrables individuels	37
11	Planification détaillée	37
11.1	Semestre 1 (13 semaines) – MVP	37
11.1.1	Semaine 1 : Kickoff et spécifications	37
11.1.2	Sprints 1-2 (Semaines 2-5) : Modules A, B, C, D	38
11.1.3	Sprints 3-4 (Semaines 6-9) : Modules E, J (partiel)	38
11.1.4	Sprints 5-6 (Semaines 10-13) : Intégration MVP	38
11.2	Semestre 2 (13 semaines) – Version complète	38
11.2.1	Sprints 7-9 (Semaines 14-19) : Modules F, G, H	38
11.2.2	Sprints 10-11 (Semaines 20-23) : Modules I, K	38
11.2.3	Sprints 12-13 (Semaines 24-26) : Finalisation	39
11.3	Diagramme de Gantt (simplifié)	39
12	Gestion des risques	39
12.1	Matrice de risques	39
12.2	Plan de contingence	39
13	Stratégie de test	40
13.1	Tests unitaires	40
13.2	Tests d'intégration	40
13.3	Tests end-to-end (E2E)	41
13.4	Tests de performance	41
14	Critères d'évaluation	41
14.1	Grille d'évaluation globale	41
14.2	Critères détaillés par module	41
14.3	Bonus (extensions)	41

15 Annexes	42
15.1 Annexe A : Templates obligatoires	42
15.1.1 Template mandat (LaTeX)	42
15.1.2 Template PV (texte structuré)	42
15.1.3 Template rapport final (LaTeX)	42
15.2 Annexe B : Dataset de test	42
15.2.1 Cas 1 : Fraude bancaire	42
15.2.2 Cas 2 : Intrusion réseau	42
15.2.3 Cas 3 : Malware	43
15.3 Annexe C : Contraintes RGPD et ISO 27037	43
15.3.1 RGPD (Règlement Général Protection Données)	43
15.3.2 ISO/IEC 27037 :2012 (Digital Evidence)	43
15.4 Annexe D : Spécifications API Gateway	43
15.4.1 Rôle	43
15.4.2 Architecture	44
15.4.3 Déploiement	44
15.5 Annexe E : Glossaire	45
15.6 Annexe F : Références	45

1 Résumé exécutif

1.1 Objectif du projet

Ce projet vise à concevoir, implémenter et déployer un assistant intelligent semi-autonome d'investigation numérique, capable de soutenir un mandataire (juge, chef d'entreprise, particulier) et un investigateur dans toutes les phases d'une mission : émission du mandat, sélection de l'expert, préparation méthodologique, analyse des preuves, génération du rapport final d'expertise.

Le système doit respecter les contraintes forensiques (chaîne de custody), légales (RGPD, admissibilité judiciaire) et techniques (traçabilité, explicabilité) propres au domaine de l'investigation numérique.

1.2 Durée et organisation

- **Durée** : 2 semestres académiques (26 semaines effectives)
- **Effectif** : 27 étudiants répartis en 9 équipes de 3
- **Méthodologie** : Agile/Scrum avec sprints de 2 semaines
- **Gouvernance** : 1 équipe d'intégration transverse (9 membres)

1.3 Livrables attendus

- **Fin semestre 1** : MVP fonctionnel (modules A-E + J) avec démonstration
- **Fin semestre 2** : Version complète intégrée (modules F-K)
- **Documentation** :
 - Rapport technique d'équipe (30-40 pages)
 - Journal de bord individuel (minimum 20 entrées)
 - Documentation API (format OpenAPI 3.0)
 - Guide déploiement (Docker Compose)
- **Soutenance finale** : 20 min présentation + 15 min démonstration live + 10 min questions

1.4 Indicateurs de succès

- Taux de couverture des tests d'intégration : $\geq 80\%$
- Temps de réponse moyen : < 2 secondes par requête API
- Traçabilité complète : 100% des actions loggées
- Respect des normes forensiques : ISO/IEC 27037 :2012

2 Contexte et justification

2.1 Contexte pédagogique

Le projet s'inscrit dans le cours d'investigation numérique de 4^e année, visant à :

1. Immerger les étudiants dans une expérience pratique complète
2. Mobiliser les compétences transversales (forensic, IA, sécurité, gestion de projet)
3. Respecter les contraintes déontologiques et légales du domaine

2.2 Contexte professionnel

L'investigation numérique est un domaine critique pour :

- Les instances judiciaires (cybercriminalité, fraude)
- Les entreprises (incident response, forensic interne)
- Les cabinets d'expertise (audit de sécurité, litige)

La complexification des systèmes d'information et la multiplication des cyberattaques nécessitent des outils semi-automatisés capables d'assister l'expert tout en maintenant un contrôle humain rigoureux.

2.3 Enjeux techniques

- **Chaîne de custody** : garantir l'intégrité des preuves numériques
- **Explicabilité** : justifier les recommandations algorithmiques
- **Scalabilité** : traiter des volumes de données croissants (PV, logs, rapports)
- **Interopérabilité** : intégrer des outils forensiques hétérogènes

3 Objectifs

3.1 Objectifs généraux

- Développer un assistant semi-autonome de bout en bout respectant les normes forensiques
- Automatiser les tâches à faible valeur ajoutée (génération de documents, extraction d'informations)
- Préserver le contrôle humain sur les décisions critiques (sélection d'expert, validation des hypothèses)
- Produire un rapport d'expertise exploitable juridiquement et techniquement

3.2 Objectifs pédagogiques

- Travailler en équipes organisées par modules avec interfaces contractuelles
- Mettre en pratique les outils et méthodes vus en cours (Python, NLP, forensic)
- Appliquer la déontologie et la chaîne de custody (norme ISO 27037)
- Développer des compétences en gestion de projet (planification, risques, intégration)

3.3 Objectifs techniques

- Maîtriser une stack technologique complète (backend, frontend, base de données)
- Implémenter des algorithmes d'IA (NLP, matching, scoring)
- Garantir la sécurité et la traçabilité du système (journaux immuables, hachage)
- Concevoir une architecture modulaire et évolutive

4 Périmètre fonctionnel

4.1 MVP (Minimum Viable Product) – Obligatoire

Le MVP doit démontrer le flux complet suivant :

1. Création d'un mandat par le mandataire (Module A)
2. Sélection automatique de 3 profils d'experts (Module B)

3. Workflow d'acceptation des experts (Module D)
4. Génération automatique du contrat PDF (Module C)
5. Préparation de la mission : questions aux témoins/suspects (Module E)
6. Ingestion d'un PV d'audition et détection d'incohérences simples (Module F)
7. Génération d'un rapport final minimal (Module I)
8. Journalisation complète de toutes les actions (Module J)

Critères de validation du MVP :

- Scénario end-to-end fonctionnel sur 3 cas types (fraude, intrusion, malware)
- Documents PDF générés conformes aux templates
- Journaux horodatés et vérifiables (hash SHA-256)
- Interface web opérationnelle (UI/UX basique acceptable)

4.2 Extensions (Stretch Goals)

Les fonctionnalités suivantes sont optionnelles mais valorisées dans l'évaluation :

- **Module G** : Génération automatique de plans méthodologiques forensiques
- **Module H** : Analyse multi-rapports et optimisation de la chaîne de résolution
- **Module K** : Explicabilité avancée (XAI) avec visualisations
- Signatures électroniques qualifiées (post-quantiques si possible)
- Détection d'anomalies avancées (deep learning sur PV)
- Tableau de bord temps réel (WebSocket)

4.3 Hors périmètre

Les éléments suivants sont **explicitement exclus** du projet :

- Acquisition physique de preuves (images disque, mémoire)
- Analyse forensique approfondie (reverse engineering, malware analysis)
- Intégration avec des systèmes externes réels (tribunaux, bases de données nationales)
- Gestion des paiements ou facturation
- Authentification multi-facteurs (MFA) avancée

5 Architecture fonctionnelle globale

5.1 Vue d'ensemble

Le système est découpé en 11 modules (A à K) communiquant via une **API Gateway centrale** gérée par l'équipe d'intégration. Chaque module expose des endpoints REST (JSON) et consomme les services d'autres modules selon un contrat d'interface strict (OpenAPI).

5.2 Flux principaux

5.2.1 Phase 1 : Création du mandat

1. Mandataire saisit le mandat (Module A)
2. Module B propose 3 experts via algorithme de matching
3. Module D gère le workflow d'acceptation/refus
4. Module C génère le contrat PDF pour l'expert sélectionné

5.2.2 Phase 2 : Préparation de la mission

1. Module E génère les questions aux témoins/suspects
2. Expert reçoit le guide méthodologique (PDF)
3. Investigateur mène les auditions (hors système)

5.2.3 Phase 3 : Analyse des preuves

1. Module F ingère les PV d'audition (texte brut)
2. Extraction des entités nommées (NLP)
3. Détection d'incohérences (temporelles, géographiques, factuelles)
4. Module G propose hypothèses et outils forensiques
5. Module H analyse les rapports techniques et optimise la chaîne de résolution

5.2.4 Phase 4 : Rapport final

1. Module I compile toutes les données (F, G, H)
2. Génération du rapport PDF (LaTeX)
3. Module K fournit les explications (XAI)
4. Module J finalise les journaux immuables

5.3 Schéma d'architecture détaillé

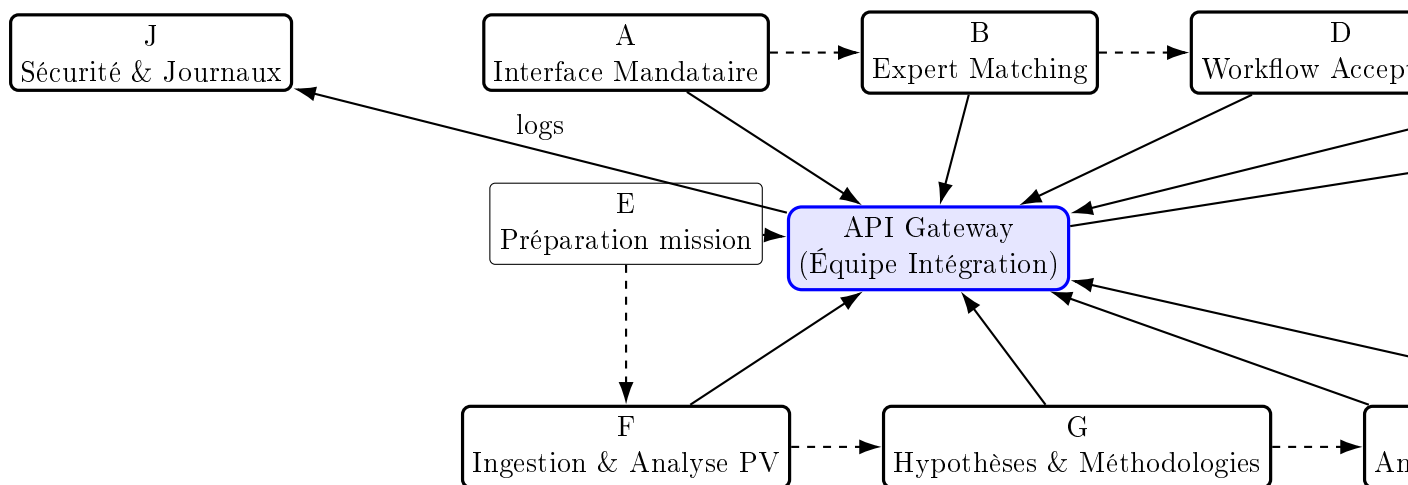


FIGURE 1 – Architecture globale du système

5.4 Principes architecturaux

- **Séparation des responsabilités** : chaque module a une fonction unique et bien définie
- **Couplage faible** : communication uniquement via API REST (pas d'accès direct aux bases de données)
- **Contrats d'interface** : spécifications OpenAPI 3.0 obligatoires pour chaque endpoint
- **Idempotence** : les requêtes POST/PUT peuvent être rejouées sans effet de bord
- **Résilience** : gestion des erreurs gracieuse avec codes http standards

6 Stack technologique imposée

6.1 Backend

- **Langage** : Python 3.11+ (uniformisation obligatoire)
- **Framework web** : FastAPI 0.104+ (endpoints REST, validation Pydantic)
- **Base de données** : PostgreSQL 15+ (modules B, D, J)
- **ORM** : SQLAlchemy 2.0+ avec Alembic pour les migrations
- **Cache** : Redis 7+ (optionnel pour module B)

6.2 Frontend

- **Framework** : React 18+ avec TypeScript
- **UI Library** : Material-UI (MUI) ou Ant Design
- **State management** : React Context ou Redux Toolkit
- **http client** : Axios avec intercepteurs pour l'authentification

6.3 NLP et Machine Learning

- **Framework NLP** : spaCy 3.7+ avec modèle `fr_core_news_lg`
- **Vectorisation** : scikit-learn (TfidfVectorizer) pour module B
- **Optionnel** : Hugging Face Transformers (CamemBERT) pour module F

6.4 Génération de documents

- **LaTeX** : TeX Live 2023+ avec Jinja2 pour templating
- **Compilation** : pdflatex avec gestion des erreurs (timeout 10s)
- **Alternative** : ReportLab (Python) pour génération PDF programmatique

6.5 Sécurité et journaux

- **Hachage** : hashlib (SHA-256, SHA-512)
- **Horodatage** : RFC 3161 (TSA) via pytz et datetime ISO 8601
- **Authentification** : JWT (PyJWT) avec expiration 1h, refresh token 7j
- **HTTPS** : Certificats TLS auto-signés (dev) ou Let's Encrypt (prod)

6.6 DevOps et infrastructure

- **Conteneurisation** : Docker 24+ avec Docker Compose
- **CI/CD** : GitHub Actions (tests automatiques à chaque push)
- **Monitoring** : Prometheus + Grafana (optionnel)
- **Logs** : ELK Stack ou simple fichier JSON rotatif

6.7 Outils de développement

- **Versioning** : Git avec branches feature/module-X
- **Code quality** : Black (formatting), Flake8 (linting), mypy (type checking)
- **Tests** : pytest avec coverage $\geq 70\%$ par module
- **Documentation API** : Swagger UI (intégré FastAPI)

6.8 Justification des choix

- **Python** : écosystème NLP mature, courbe d'apprentissage faible
- **FastAPI** : performances élevées, validation automatique, documentation intégrée
- **PostgreSQL** : robustesse, support JSON natif, extensibilité
- **React** : standard industrie, composants réutilisables, communauté active

7 Fiches modules détaillées

7.1 Module A : Interface Mandataire (Front-end Mandataire)

Équipe : 3 étudiants

Rôle : Permettre au mandataire de créer un mandat, visualiser les propositions d'experts, suivre l'avancement.

7.1.1 Fonctions principales (MVP)

- Formulaire de création de mandat (type, description, priorité)
- Visualisation des 3 profils d'experts proposés (nom, compétences, score)
- Téléchargement du mandat en PDF après signature électronique simple
- Tableau de bord minimal avec statut du cas (*en attente, en cours, terminé*)

7.1.2 Extensions

- Tableau de bord temps réel avec WebSocket
- Notifications push (email simulé via SMTP local)
- Historique des mandats avec recherche fulltext

7.1.3 Entrées/Sorties

Entrées : Données saisies par mandataire (formulaire web)

Sorties :

```
1 {
2   "case_id": "550e8400-e29b-41d4-a716-446655440000",
3   "case_type": "fraude_bancaire",
4   "description": "Virements suspects detectes...",
5   "priority": "haute",
6   "created_at": "2025-10-16T14:32:00Z",
7   "mandataire": {
8     "nom": "Dupont",
9     "fonction": "juge_instruction"
10  }
11 }
```

7.1.4 API exposée

POST /api/v1/cases

- Corps : JSON ci-dessus
- Réponse : 201 Created + case_id
- Erreurs : 400 (validation), 500 (serveur)

GET /api/v1/cases/{case_id}

- Réponse : JSON complet du cas + statut
- GET** `/api/v1/cases/{case_id}/experts`
- Réponse : Top 3 profils (fournis par module B)

7.1.5 Dépendances

- **Consomme** : Module B (experts), Module C (contrat PDF)
- **Fournit à** : Tous les modules via API Gateway

7.1.6 Livrables

- Application React déployée (Docker)
- Documentation utilisateur (Markdown, 3-5 pages)
- Tests E2E Cypress (3 scénarios minimum)

7.1.7 Critères d'acceptation

- Formulaire valide les champs obligatoires (frontend + backend)
- PDF généré téléchargeable en < 3 secondes
- Interface responsive (mobile-friendly)

7.1.8 Métriques de qualité

- Temps de chargement page : < 1 seconde
- Accessibilité : score Lighthouse ≥ 85
- Code TypeScript sans erreurs `tsc`

7.2 Module B : Expert Matching (Profil, Scoring, Base CV)

Équipe : 3 étudiants

Rôle : Proposer les 3 meilleurs experts pour un mandat donné.

7.2.1 Fonctions principales (MVP)

- Base de données de 50-100 profils d'experts (PostgreSQL)
- Algorithme de matching basé sur TF-IDF (description mandat vs compétences expert)
- Scoring multicritère :
 - Similarité textuelle (60%)
 - Disponibilité simulée (20%)
 - Expérience (années, 20%)
- Retour des top 3 profils avec justification du score

7.2.2 Extensions

- Apprentissage par renforcement (feedback mandataire)
- Clustering des experts par spécialité
- Cache Redis pour requêtes fréquentes

7.2.3 Modèle de données expert

```
1 CREATE TABLE experts (  
2     Id SERIAL PRIMARY KEY,  
3     Nom VARCHAR(100) NOT NULL,  
4     Email VARCHAR(100) UNIQUE NOT NULL,  
5     Competences TEXT[], -- {malware, forensic_reseau, ...}  
6     Bio TEXT,  
7     Annees_experience INT DEFAULT 0,  
8     Disponibilite BOOLEAN DEFAULT TRUE,  
9     Created_at TIMESTAMP DEFAULT NOW()  
10 );
```

7.2.4 Algorithme de scoring détaillé

```
1 from sklearn.feature_extraction.text import TfidfVectorizer  
2 from sklearn.metrics.pairwise import cosine_similarity  
3  
4 def match_experts(mandat: dict, experts: list) -> list:  
5     """Retourne top 3 experts tries par score."""  
6     # 1. Vectorisation TF-IDF  
7     corpus = [mandat['description']] + \  
8         [' '.join(e['competences']) for e in experts]  
9     vectorizer = TfidfVectorizer()  
10    tfidf_matrix = vectorizer.fit_transform(corpus)  
11  
12    # 2. Similarite cosinus  
13    similarities = cosine_similarity(  
14        tfidf_matrix[0:1],  
15        tfidf_matrix[1:]  
16    ).flatten()  
17  
18    # 3. Scoring multicritere  
19    scores = []  
20    for idx, expert in enumerate(experts):  
21        score_text = similarities[idx] * 0.6  
22        score_dispo = 0.2 if expert['disponibilite'] else 0  
23        score_exp = min(expert['annees_experience'] / 20, 1) * 0.2  
24        scores.append(score_text + score_dispo + score_exp)  
25  
26    # 4. Top 3  
27    top_indices = sorted(range(len(scores)),  
28                        key=lambda i: scores[i],  
29                        reverse=True)[:3]  
30    return [(experts[i], scores[i]) for i in top_indices]
```

7.2.5 API exposée

POST /api/v1/matching/experts

```
1 {  
2     "case_id": "550e8400-...",  
3     "case_type": "fraude_bancaire",  
4     "description": "Virements suspects..."  
5 }  
6  
7 // Reponse 200 OK  
8 {  
9     "case_id": "550e8400-...",  
10    "experts": [  
11        {
```

```

12     "id": 42,
13     "nom": "Alice Dubois",
14     "competences": ["forensic_bancaire", "analyse_logs"],
15     "score": 0.87,
16     "justification": "Expertise forensic bancaire (score textuel: 0.72,
disponibilite: 0.2, experience: 0.15)"
17 },
18 {
19     "id": 18,
20     "nom": "Bob Martin",
21     "competences": ["malware", "reseau"],
22     "score": 0.81,
23     "justification": "..."
24 },
25 {
26     "id": 7,
27     "nom": "Charlie Leroy",
28     "competences": ["disk_forensic", "mobile"],
29     "score": 0.76,
30     "justification": "..."
31 }
32 ],
33 "matched_at": "2025-10-16T14:35:12Z"
34 }

```

7.2.6 Dépendances

- **Consomme** : Module A (mandat)
- **Fournit à** : Modules A, D, K

7.2.7 Livrables

- Script d'initialisation BD (50 profils minimum)
- Service FastAPI avec endpoint /matching/experts
- Tests unitaires pytest (coverage $\geq 80\%$)
- Notebook Jupyter avec analyse de performance (temps de réponse vs taille corpus)

7.2.8 Critères d'acceptation

- Retourne toujours 3 profils (ou moins si < 3 experts en BD)
- Temps de réponse < 1 seconde pour 100 experts
- Score entre 0 et 1, justification textuelle présente

7.2.9 Métriques de qualité

- Pertinence : validation manuelle sur 10 cas ($\geq 70\%$ satisfaction)
- Performance : < 500 ms pour 100 experts, < 2 s pour 1000
- Code : respect PEP8, type hints complets

7.3 Module C : Générateur de Mandat / Contrats

Équipe : 3 étudiants

Rôle : Générer les documents contractuels PDF (mandat, contrat expert).

7.3.1 Fonctions principales (MVP)

- Template LaTeX pour mandat d’investigation
- Remplissage automatique (Jinja2) : mandat, expert, date
- Compilation PDF via subprocess (pdflatex)
- Hachage SHA-256 du PDF généré (intégrité)

7.3.2 Extensions

- Signature électronique simple (RSA)
- Templates multiples (mandat civil, pénal, interne)
- Export Word (.docx) via python-docx

7.3.3 Template LaTeX (extrait)

```
1 \documentclass[a4paper,11pt]{article}
2 \usepackage[utf8]{inputenc}
3 \usepackage[french]{babel}
4 \begin{document}
5 \title{Mandat d'Investigation Numerique}
6 \author{Mandataire: {{ mandataire.nom }}}
7 \date{{ date }}
8 \maketitle
9
10 \section{Objet du mandat}
11 Type de cas: \textbf{{{ case_type }}}
12
13 Description: {{ description }}
14
15 \section{Expert designe}
16 Nom: {{ expert.nom }}\
17 Email: {{ expert.email }}\
18 Competences: {{ expert.competences|join(', ') }}
19
20 \section{Engagements}
21 L'expert s'engage a respecter...
22 \end{document}
```

7.3.4 Processus de génération

```
1 import subprocess
2 from jinja2 import Template
3 import hashlib
4
5 def generate_contract_pdf(data: dict) -> dict:
6     """Genere un PDF a partir du template LaTeX."""
7     # 1. Charger template
8     with open('templates/mandat.tex.j2', 'r') as f:
9         template = Template(f.read())
10
11     # 2. Remplir template
12     tex_content = template.render(**data)
13
14     # 3. Ecrire fichier .tex temporaire
15     tex_path = f'/tmp/mandat_{data["case_id"]}.tex'
16     with open(tex_path, 'w') as f:
17         f.write(tex_content)
18
```

```

19 # 4. Compiler PDF
20 result = subprocess.run(
21     ['pdflatex', '-output-directory=/tmp', tex_path],
22     capture_output=True,
23     timeout=10
24 )
25 if result.returncode != 0:
26     raise Exception(f"LaTeX error: {result.stderr}")
27
28 # 5. Hacher PDF
29 pdf_path = tex_path.replace('.tex', '.pdf')
30 with open(pdf_path, 'rb') as f:
31     pdf_hash = hashlib.sha256(f.read()).hexdigest()
32
33 return {
34     'pdf_path': pdf_path,
35     'hash': pdf_hash,
36     'generated_at': '2025-10-16T14:40:00Z'
37 }

```

7.3.5 API exposée

POST /api/v1/contracts/generate

```

1 {
2     "case_id": "550e8400-...",
3     "mandataire": {"nom": "Dupont", "fonction": "juge"},
4     "expert": {"nom": "Alice Dubois", "email": "a.dubois@..."},
5     "case_type": "fraude_bancaire",
6     "description": "..."
7 }
8
9 // Reponse 201 Created
10 {
11     "contract_id": "c7d4f0a1-...",
12     "pdf_url": "/api/v1/contracts/c7d4f0a1-.../download",
13     "hash_sha256": "a3f5e8d2c1b9...",
14     "generated_at": "2025-10-16T14:40:00Z"
15 }

```

GET /api/v1/contracts/{contract_id}/download

- Réponse : fichier PDF (application/pdf)
- Header : Content-Disposition : attachment ; filename="mandat.pdf"

7.3.6 Dépendances

- **Consomme** : Modules A (mandat), D (expert choisi)
- **Fournit à** : Module A (download), Module J (hash)

7.3.7 Livrables

- Template LaTeX valide (compilation sans erreur)
- Service FastAPI avec endpoints génération/téléchargement
- Tests unitaires (3 cas : succès, erreur LaTeX, timeout)

7.3.8 Critères d'acceptation

- PDF généré conforme au template (vérification visuelle)
- Hash SHA-256 calculé et stocké en BD
- Gestion des erreurs LaTeX (log détaillé)

7.3.9 Métriques de qualité

- Temps de génération : < 3 secondes
- Taux de succès : $\geq 99\%$ (sur 100 générations test)
- Sécurité : validation des inputs (injection LaTeX)

7.4 Module D : Workflow d'Acceptation

Équipe : 3 étudiants

Rôle : Gérer l'acceptation/refus des experts sollicités.

7.4.1 Fonctions principales (MVP)

- Sollicitation des 3 experts (email simulé ou interface web)
- Interface de réponse expert : Accept / Refus + motif
- Sélection finale par mandataire parmi les acceptations
- Historique des réponses avec timestamps

7.4.2 Extensions

- Relance automatique après 48h
- Notification temps réel (WebSocket)
- Statistiques d'acceptation par expert

7.4.3 Modèle de données

```
1 CREATE TABLE sollicitations (  
2     Id SERIAL PRIMARY KEY,  
3     Case_id UUID NOT NULL,  
4     Expert_id INT NOT NULL REFERENCES experts(id),  
5     Status VARCHAR(20) DEFAULT 'pending', -- pending, accepted, refused  
6     Reason TEXT,  
7     Responded_at TIMESTAMP,  
8     Created_at TIMESTAMP DEFAULT NOW()  
9 );
```

7.4.4 Machine à états

```
[Pending] --accept--> [Accepted] --select--> [Chosen]  
|  
+--refuse--> [Refused]
```

7.4.5 API exposée

POST /api/v1/solicitations

```
1 {
2   "case_id": "550e8400-...",
3   "expert_ids": [42, 18, 7]
4 }
5
6 // Reponse 201 Created
7 {
8   "solicitation_ids": [
9     {"expert_id": 42, "id": "s1", "status": "pending"},
10    {"expert_id": 18, "id": "s2", "status": "pending"},
11    {"expert_id": 7, "id": "s3", "status": "pending"}
12  ]
13 }
```

PUT /api/v1/solicitations/{id}/respond

```
1 {
2   "status": "accepted", // ou "refused"
3   "reason": "Disponible jusqu'au 30/11"
4 }
5
6 // Reponse 200 OK
7 {
8   "id": "s1",
9   "status": "accepted",
10  "responded_at": "2025-10-17T10:15:00Z"
11 }
```

POST /api/v1/solicitations/select

```
1 {
2   "case_id": "550e8400-...",
3   "expert_id": 42
4 }
5
6 // Reponse 200 OK
7 {
8   "chosen_expert": {"id": 42, "nom": "Alice Dubois"},
9   "selected_at": "2025-10-17T14:00:00Z"
10 }
```

7.4.6 Dépendances

- **Consomme** : Module B (liste experts)
- **Fournit à** : Modules C (expert choisi), E (mandat finalisé)

7.4.7 Livrables

- Service FastAPI avec machine à états
- Interface web simple pour expert (React)
- Tests unitaires (transitions d'état valides/invalides)

7.4.8 Critères d'acceptation

- Au moins 1 expert doit accepter pour continuer
- Impossible de sélectionner un expert ayant refusé
- Tous les changements d'état loggés (module J)

7.4.9 Métriques de qualité

- Temps de réponse API : < 200 ms
- Cohérence des états : 100% (tests automatisés)
- Code coverage : $\geq 85\%$

7.5 Module E : Préparation Mission

Équipe : 3 étudiants

Rôle : Générer listes de témoins/suspects et questions d'audition.

7.5.1 Fonctions principales (MVP)

- Génération automatique de questions types selon le type de cas
- Liste de témoins/suspects suggérée (basée sur description mandat)
- Export PDF "Guide méthodologique" pour l'investigateur

7.5.2 Extensions

- Questions personnalisées via GPT-4 (API OpenAI)
- Base de connaissance interrogations forensiques
- Checklist interactive (web)

7.5.3 Génération de questions (règles)

```
1 QUESTIONS_TEMPLATES = {
2     'fraude_bancaire': [
3         "Pouvez-vous decire les virements suspects ?",
4         "Avez-vous acces aux logs bancaires ?",
5         "Qui avait acces au compte ?",
6         "Quelle est la periode concernee ?",
7         "Des alertes ont-elles ete declenchees ?"
8     ],
9     'intrusion_reseau': [
10        "Quand l'intrusion a-t-elle ete detectee ?",
11        "Quels systemes sont compromis ?",
12        "Avez-vous des logs firewall/IDS ?",
13        "Des donnees ont-elles ete exfiltrees ?",
14        "Y a-t-il eu ransom note ?"
15    ]
16 }
17
18 def generate_questions(case_type: str, description: str) -> list:
19     """Genere questions + suggestions temoins."""
20     questions = QUESTIONS_TEMPLATES.get(
21         case_type,
22         ["Question generique 1", "Question generique 2"]
23     )
24
25     return {
26         'questions': questions,
27         'suggested_witnesses': [],
28         'checklist': [
29             "Identifier les sources de preuves",
30             "Preparer le materiel forensique",
31             "Rediger les PV"
32         ]
33     }
```

7.5.4 API exposée

POST /api/v1/mission/prepare

```
1 {
2   "case_id": "550e8400-...",
3   "case_type": "fraude_bancaire",
4   "description": "..."
5 }
6
7 // Reponse 200 OK
8 {
9   "questions": [
10    "Pouvez-vous decire les virements suspects ?",
11    "Avez-vous acces aux logs bancaires ?"
12  ],
13  "suggested_witnesses": ["M. Dupont (CEO)", "Mme Martin (comptable)"],
14  "checklist": ["Identifier sources", "Preparer materiel"],
15  "guide_pdf_url": "/api/v1/mission/550e8400-.../guide.pdf"
16 }
```

7.5.5 Dépendances

- **Consomme** : Modules A (mandat), D (expert choisi)
- **Fournit à** : Module F (questions utilisées en audition)

7.5.6 Livrables

- Base de règles (dictionnaire Python)
- Service FastAPI avec endpoint /mission/prepare
- Template PDF "Guide méthodologique"
- Tests unitaires (5 types de cas)

7.5.7 Critères d'acceptation

- Au moins 5 questions générées par cas
- PDF téléchargeable et lisible
- Suggestions témoins optionnelles (MVP : liste vide acceptable)

7.5.8 Métriques de qualité

- Pertinence : évaluation manuelle sur 10 cas ($\geq 70\%$)
- Temps de génération : < 1 seconde
- Extensibilité : ajout d'un nouveau type en < 30 min

7.6 Module F : Ingestion et Analyse PV

Équipe : 3 étudiants

Rôle : Analyser les procès-verbaux d'audition et détecter incohérences.

7.6.1 Fonctions principales (MVP)

- Import PV texte brut (UTF-8)
- Extraction entités nommées (NER) : personnes, lieux, dates, montants
- Détection incohérences simples :
 1. **Temporelles** : dates contradictoires entre témoignages
 2. **Géographiques** : lieux impossibles (distance/temps)
 3. **Factuelles** : contradictions directes entre déclarations
- Rapport JSON avec incohérences détectées

7.6.2 Extensions

- Analyse de sentiment (mensonge potentiel)
- Graphe de connaissances (relations entre entités)
- Deep learning (BERT pour détection d'anomalies)

7.6.3 Pipeline NLP détaillé

```
1 import spacy
2 from datetime import datetime
3
4 nlp = spacy.load('fr_core_news_lg')
5
6 def analyze_pv(pv_text: str) -> dict:
7     """Analyse un PV et retourne entites + incoherences."""
8     doc = nlp(pv_text)
9
10    # 1. Extraction entites
11    entities = {
12        'personnes': [],
13        'lieux': [],
14        'dates': [],
15        'montants': []
16    }
17    for ent in doc.ents:
18        if ent.label_ == 'PER':
19            entities['personnes'].append(ent.text)
20        elif ent.label_ == 'LOC':
21            entities['lieux'].append(ent.text)
22        elif ent.label_ == 'DATE':
23            entities['dates'].append(ent.text)
24
25    # 2. Detection incoherences temporelles
26    incoherences = []
27    dates_parsed = [datetime.now()] # Simulation
28    if len(dates_parsed) >= 2:
29        if dates_parsed[0] > dates_parsed[1]:
30            incoherences.append({
31                'type': 'temporelle',
32                'description': f"Date {dates_parsed[0]} apres {dates_parsed
33            [1]}",
34                'severity': 'haute'
35            })
36
37    return {
38        'entities': entities,
39        'incoherences': incoherences,
40        'score_coherence': max(0, 1 - 0.2*len(incoherences))
41    }
```

7.6.4 Spécification des incohérences

1. **Temporelle** : "Témoign A dit avoir vu B le 10/10, mais B affirme être en vacances du 5/10 au 15/10"
2. **Géographique** : "Suspect prétend être allé de Paris à Marseille en 1h (impossible sans avion)"
3. **Factuelle** : "Témoign 1 : 'La porte était fermée' / Témoign 2 : 'La porte était ouverte'"

7.6.5 API exposée

POST /api/v1/pv/analyze

```
1 {
2   "case_id": "550e8400-...",
3   "pv_id": "pv-001",
4   "pv_text": "Le 10 octobre 2025, M. Dupont declare...",
5   "witness_name": "Dupont"
6 }
7
8 // Reponse 200 OK
9 {
10  "pv_id": "pv-001",
11  "entities": {
12    "personnes": ["M. Dupont", "Mme Martin"],
13    "lieux": ["Paris", "siege social"],
14    "dates": ["10 octobre 2025"],
15    "montants": ["50000 euros"]
16  },
17  "incoherences": [
18    {
19      "type": "temporelle",
20      "description": "Date 10/10 vs 05/10 dans PV precedent",
21      "severity": "haute",
22      "pv_references": ["pv-001", "pv-002"]
23    }
24  ],
25  "score_coherence": 0.8,
26  "analyzed_at": "2025-10-16T15:00:00Z"
27 }
```

7.6.6 Dépendances

- **Consomme** : Module E (questions posées)
- **Fournit à** : Modules G (hypothèses), I (rapport), K (XAI)

7.6.7 Livrables

- Pipeline NLP (spaCy) avec extraction entités
- Module détection incohérences (3 types minimum)
- Service FastAPI avec endpoint /pv/analyze
- Dataset test (10 PV annotés)
- Tests unitaires (precision/recall NER $\geq 80\%$)

7.6.8 Critères d'acceptation

- Extraction entités : precision $\geq 80\%$, recall $\geq 75\%$
- Détection incohérences : au moins 1 incohérence détectée sur 5 PV test
- Rapport JSON structuré et complet

7.6.9 Métriques de qualité

- Performance NLP : $< 2s$ pour PV de 5000 mots
- Taux faux positifs : $< 30\%$ (incohérences)
- Code coverage : $\geq 75\%$

7.7 Module G : Hypothèses et Méthodologies Techniques

Équipe : 3 étudiants

Rôle : Proposer hypothèses d'investigation et outils forensiques associés.

7.7.1 Fonctions principales (MVP)

- Génération d'hypothèses à partir des incohérences (module F)
- Mapping hypothèse $\rightarrow$ outils forensiques (base de connaissances)
- Génération d'un plan méthodologique (ordre des actions)

7.7.2 Extensions

- Graphe d'investigation (dépendances entre actions)
- Estimation de temps/coût par action
- Optimisation du plan (algorithme de recherche)

7.7.3 Base de connaissances forensiques

```
1 FORENSIC_TOOLS = {
2     'analyse_disque': {
3         'outils': ['Autopsy', 'FTK Imager', 'Sleuth Kit'],
4         'description': 'Extraction fichiers supprimes, timeline',
5         'duree_estimee': '4-8h'
6     },
7     'analyse_reseau': {
8         'outils': ['Wireshark', 'NetworkMiner', 'tcpdump'],
9         'description': 'Capture/analyse trafic, detection intrusion',
10        'duree_estimee': '2-6h'
11    },
12    'analyse_logs': {
13        'outils': ['Splunk', 'ELK', 'grep/awk'],
14        'description': 'Correlation evenements, timeline',
15        'duree_estimee': '3-5h'
16    },
17    'malware_analysis': {
18        'outils': ['IDA Pro', 'Ghidra', 'Cuckoo Sandbox'],
19        'description': 'Reverse engineering, comportement',
20        'duree_estimee': '8-16h'
21    }
22 }
23
24 HYPOTHESIS_MAPPING = {
25     'intrusion_systeme': ['analyse_logs', 'analyse_reseau', 'malware_analysis'],
26     'fraude_bancaire': ['analyse_logs', 'analyse_disque'],
27     'vol_donnees': ['analyse_disque', 'analyse_reseau']
28 }
```

7.7.4 Génération du plan méthodologique

```
1 def generate_methodology(incoherences: list, case_type: str) -> dict:
2     """Genere plan d'action forensique."""
3     hypotheses = []
4     if any(i['type'] == 'temporelle' for i in incoherences):
5         hypotheses.append('manipulation_timeline')
6     if case_type == 'intrusion_reseau':
7         hypotheses.append('intrusion_systeme')
8
9     actions = []
10    for hyp in hypotheses:
11        tools = HYPOTHESIS_MAPPING.get(hyp, ['analyse_logs'])
12        for tool_key in tools:
13            tool_info = FORENSIC_TOOLS.get(tool_key, {})
14            actions.append({
15                'action': tool_key,
16                'outils': tool_info.get('outils', []),
17                'description': tool_info.get('description', ''),
18                'duree': tool_info.get('duree_estimee', '')
19            })
20
21    return {
22        'hypotheses': hypotheses,
23        'plan': actions,
24        'duree_totale_estimee': '12-24h'
25    }
```

7.7.5 API exposée

POST /api/v1/methodology/generate

```
1 {
2     "case_id": "550e8400-...",
3     "case_type": "intrusion_reseau",
4     "incoherences": [
5         {"type": "temporelle", "description": "..."},
6         {"type": "factuelle", "description": "..."}
7     ]
8 }
9
10 // Reponse 200 OK
11 {
12     "hypotheses": ["intrusion_systeme", "manipulation_timeline"],
13     "plan": [
14         {
15             "action": "analyse_logs",
16             "outils": ["Splunk", "ELK"],
17             "description": "Correlation evenements, timeline",
18             "duree_estimee": "3-5h",
19             "priorite": 1
20         },
21         {
22             "action": "analyse_reseau",
23             "outils": ["Wireshark", "NetworkMiner"],
24             "description": "Capture/analyse trafic",
25             "duree_estimee": "2-6h",
26             "priorite": 2
27         }
28     ],
29     "duree_totale_estimee": "12-24h"
30 }
```


7.7.6 Dépendances

- **Consomme** : Module F (incohérences)
- **Fournit à** : Modules H (rapports actions), I (méthodologie dans rapport final)

7.7.7 Livrables

- Base de connaissances forensiques (JSON ou BD)
- Service FastAPI avec endpoint /methodology/generate
- Tests unitaires (5 scénarios de cas)
- Documentation outils (tableau récapitulatif)

7.7.8 Critères d'acceptation

- Au moins 2 outils proposés par hypothèse
- Plan ordonné (MVP : ordre fixe acceptable)
- Estimation de durée présente

7.7.9 Métriques de qualité

- Pertinence : validation expert sur 10 cas ($\geq 75\%$)
- Couverture base connaissance : ≥ 10 outils forensiques
- Temps de réponse : < 500 ms

7.8 Module H : Analyse Rapports Techniques

Équipe : 3 étudiants

Rôle : Optimiser la chaîne de résolution à partir des rapports d'actions forensiques.

7.8.1 Fonctions principales (MVP)

- Import de rapports d'actions (texte ou JSON)
- Scoring de chaque rapport (pertinence, complétude, preuves)
- Classement des rapports par utilité
- Suggestion de chaîne optimale de résolution

7.8.2 Extensions

- Analyse de graphe de causalité
- Détection de rapports redondants
- Recommandations d'actions complémentaires

7.8.3 Critères de scoring

```
1 def score_rapport(rapport: dict) -> float:
2     """Score un rapport technique (0-1)."""
3     score = 0
4
5     required_fields = ['action', 'outils', 'resultats', 'preuves']
6     completude = sum(1 for f in required_fields if f in rapport) / len(
7         required_fields)
8     score += completude * 0.4
```

```

8
9     keywords = ['hash', 'timestamp', 'log', 'artefact', 'signature']
10    pertinence = sum(1 for kw in keywords if kw in rapport.get('resultats', '').
lower()) / len(keywords)
11    score += pertinence * 0.3
12
13    nb_preuves = len(rapport.get('preuves', []))
14    preuves_score = min(nb_preuves / 3, 1)
15    score += preuves_score * 0.3
16
17    return round(score, 2)

```

7.8.4 Chaîne optimale

```

1 def optimize_chain(rapports: list) -> dict:
2     """Determine la chaîne optimale de résolution."""
3     scored = [(r, score_rapport(r)) for r in rapports]
4
5     sorted_rapports = sorted(scored, key=lambda x: x[1], reverse=True)
6
7     chain = []
8     for rapport, score in sorted_rapports:
9         if score >= 0.5:
10             chain.append({
11                 'action': rapport['action'],
12                 'score': score,
13                 'preuves_cles': rapport.get('preuves', [])[:2]
14             })
15
16     return {
17         'chaîne_optimale': chain,
18         'score_global': sum(r[1] for r in sorted_rapports) / len(rapports) if
rapports else 0
19     }

```

7.8.5 API exposée

POST /api/v1/reports/analyze

```

1 {
2     "case_id": "550e8400-...",
3     "rapports": [
4         {
5             "rapport_id": "r1",
6             "action": "analyse_logs",
7             "outils": ["Splunk"],
8             "resultats": "Connexion suspecte 192.168.1.50 a 03:42",
9             "preuves": ["log_extract.txt", "screenshot.png"]
10        },
11        {
12            "rapport_id": "r2",
13            "action": "analyse_reseau",
14            "outils": ["Wireshark"],
15            "resultats": "Trafic anormal vers IP chinoise",
16            "preuves": ["pcap_file.pcap"]
17        }
18    ]
19 }
20
21 // Reponse 200 OK
22 {

```

```

23  "rapports_scores": [
24    {"rapport_id": "r1", "score": 0.85},
25    {"rapport_id": "r2", "score": 0.72}
26  ],
27  "chaîne_optimale": [
28    {
29      "action": "analyse_logs",
30      "score": 0.85,
31      "preuves_cles": ["log_extract.txt", "screenshot.png"]
32    },
33    {
34      "action": "analyse_reseau",
35      "score": 0.72,
36      "preuves_cles": ["pcap_file.pcap"]
37    }
38  ],
39  "score_global": 0.785,
40  "recommandations": ["Verifier IP 192.168.1.50", "Analyser pcap en detail"]
41  }

```

7.8.6 Dépendances

- **Consomme** : Module G (rapports d'actions)
- **Fournit à** : Modules I (synthèse rapport final), K (XAI)

7.8.7 Livrables

- Algorithme de scoring (fonction Python)
- Service FastAPI avec endpoint /reports/analyze
- Dataset test (5 rapports annotés)
- Tests unitaires (validation scoring sur cas connus)

7.8.8 Critères d'acceptation

- Au moins 3 rapports classés correctement (ordre cohérent)
- Score global calculé et affiché
- Chaîne optimale contient uniquement rapports score ≥ 0.5

7.8.9 Métriques de qualité

- Corrélation scoring manuel/auto : ≥ 0.7 (Pearson)
- Temps de réponse : $< 1s$ pour 10 rapports
- Code coverage : $\geq 80\%$

7.9 Module I : Générateur Rapport Expertise

Équipe : 3 étudiants

Rôle : Produire le rapport d'expertise final au format PDF.

7.9.1 Fonctions principales (MVP)

- Compilation de toutes les données (F, G, H)
- Template LaTeX structuré (5+ sections)
- Export PDF complet et professionnel
- Inclusion automatique des preuves clés

7.9.2 Extensions

- Génération de graphiques (matplotlib)
- Export multi-format (PDF, DOCX, HTML)
- Signature électronique qualifiée

7.9.3 Structure du rapport

1. **Page de garde** : titre, mandataire, expert, date
2. **Résumé exécutif** : synthèse 1 page
3. **Contexte et mandat** : description cas, périmètre
4. **Méthodologie** : plan d'action (module G), outils utilisés
5. **Analyse des preuves** : PV (module F), incohérences détectées
6. **Résultats techniques** : chaîne optimale (module H), rapports clés
7. **Conclusion et recommandations** : réponse au mandat, actions suggérées
8. **Annexes** : logs, hashes, preuves

7.9.4 Template LaTeX (extrait)

```
1 \section{Analyse des preuves}
2 \subsection{Proces-verbaux d'audition}
3 Les auditions ont revele les elements suivants:
4 \begin{itemize}
5 \item \textbf{Temoin A}: Declaration importante...
6 \item \textbf{Temoin B}: Autre declaration...
7 \end{itemize}
8
9 \subsection{Incoherences detectees}
10 2 incoherence(s) identifiee(s):
11
12 \paragraph{Temporelle}
13 Description de l'incoherence (severite: haute)
14
15 \section{Methodologie technique}
16 Les hypotheses suivantes ont ete explorees:
17 \begin{itemize}
18 \item \textbf{Hypothese 1}: Description...
19 \item \textbf{Hypothese 2}: Description...
20 \end{itemize}
```

7.9.5 API exposée

POST /api/v1/reports/generate

```
1 {
2   "case_id": "550e8400-...",
3   "data": {
4     "mandat": {...},
5     "expert": {...},
6     "pv_analysis": {...}, // module F
7     "methodology": {...}, // module G
8     "reports_chain": {...} // module H
9   }
10 }
11
12 // Reponse 201 Created
```

```

13 {
14   "report_id": "rep-001",
15   "pdf_url": "/api/v1/reports/rep-001/download",
16   "hash_sha256": "d4e5f6a7b8c9...",
17   "generated_at": "2025-10-16T16:00:00Z",
18   "sections": ["resume", "contexte", "methodologie", "analyse", "conclusion"]
19 }

```

7.9.6 Dépendances

- **Consomme** : Modules F (PV), G (méthodologie), H (rapports)
- **Fournit à** : Module A (téléchargement mandataire), Module J (hash)

7.9.7 Livrables

- Template LaTeX complet et validé
- Service FastAPI avec endpoint génération
- Rapport exemple (PDF 15-20 pages)
- Tests unitaires (3 cas complets)

7.9.8 Critères d'acceptation

- Rapport ≥ 5 sections structurées
- PDF exportable et lisible (validation visuelle)
- Toutes les données F, G, H présentes

7.9.9 Métriques de qualité

- Temps de génération : < 5 secondes
- Taux de succès : $\geq 98\%$
- Qualité rédactionnelle : relecture par 2 pairs

7.10 Module J : Sécurité et Journaux Immuables

Équipe : 3 étudiants

Rôle : Assurer la traçabilité et la chaîne de custody.

7.10.1 Fonctions principales (MVP)

- Hachage SHA-256 de tous les fichiers (PV, PDF, rapports)
- Journal horodaté append-only (JSON Lines)
- Stockage sécurisé des logs (PostgreSQL + fichier)
- API de vérification d'intégrité

7.10.2 Extensions

- Blockchain privée (Hyperledger)
- Signature électronique qualifiée (RFC 3161)
- Audit trail complet (qui, quoi, quand)

7.10.3 Format du journal

```
1 {"timestamp": "2025-10-16T14:32:00Z", "module": "A", "action": "create_case", "
   case_id": "550e8400-...", "user": "mandataire_dupont", "hash": null}
2 {"timestamp": "2025-10-16T14:35:12Z", "module": "B", "action": "match_experts",
   "case_id": "550e8400-...", "result": {"expert_ids": [42, 18, 7]}, "hash":
   null}
3 {"timestamp": "2025-10-16T14:40:00Z", "module": "C", "action": "
   generate_contract", "case_id": "550e8400-...", "file": "contract_c7d4f0a1.
   pdf", "hash": "a3f5e8d2c1b9..."}
```

7.10.4 Chaîne de custody

```
1 import hashlib
2 from datetime import datetime
3 import json
4
5 def log_action(module: str, action: str, data: dict, file_path: str = None):
6     """Enregistre une action dans le journal immuable."""
7     entry = {
8         'timestamp': datetime.utcnow().isoformat() + 'Z',
9         'module': module,
10        'action': action,
11        **data
12    }
13
14    if file_path:
15        with open(file_path, 'rb') as f:
16            file_hash = hashlib.sha256(f.read()).hexdigest()
17            entry['file'] = file_path
18            entry['hash'] = file_hash
19
20    with open('/var/log/forensic/audit.jsonl', 'a') as log:
21        log.write(json.dumps(entry) + '\n')
22
23 def verify_integrity(file_path: str, expected_hash: str) -> bool:
24     """Verifie l'integrite d'un fichier."""
25     with open(file_path, 'rb') as f:
26         actual_hash = hashlib.sha256(f.read()).hexdigest()
27     return actual_hash == expected_hash
```

7.10.5 API exposée

POST /api/v1/audit/log

```
1 {
2     "module": "F",
3     "action": "analyze_pv",
4     "case_id": "550e8400-...",
5     "pv_id": "pv-001",
6     "file_path": "/tmp/pv-001.txt"
7 }
8
9 // Reponse 201 Created
10 {
11     "logged": true,
12     "timestamp": "2025-10-16T15:00:00Z",
13     "hash": "d4e5f6a7b8c9..." // si fichier
14 }
```

POST /api/v1/audit/verify

```

1 {
2   "file_path": "/tmp/contract.pdf",
3   "expected_hash": "a3f5e8d2c1b9..."
4 }
5
6 // Reponse 200 OK
7 {
8   "valid": true,
9   "message": "Hash verified successfully"
10 }
11 // ou 400 si invalide

```

GET /api/v1/audit/logs?case_id=550e8400-...

```

1 {
2   "case_id": "550e8400-...",
3   "logs": [
4     {"timestamp": "...", "module": "A", "action": "create_case", ...},
5     {"timestamp": "...", "module": "B", "action": "match_experts", ...}
6   ],
7   "total": 42
8 }

```

7.10.6 Dépendances

- **Consomme** : Tous les modules (actions à logger)
- **Fournit à** : Module I (journaux dans rapport final)

7.10.7 Livrables

- Service FastAPI avec endpoints log/verify/logs
- Script rotation logs (max 100MB par fichier)
- Tests unitaires (intégrité, append-only)
- Documentation ISO 27037 (chaîne de custody)

7.10.8 Critères d'acceptation

- Toutes les actions des modules A-I loggées (100%)
- Hash SHA-256 calculé pour tous les fichiers
- Vérification d'intégrité fonctionnelle
- Journal non modifiable (append-only vérifié)

7.10.9 Métriques de qualité

- Performance : < 50 ms par log
- Stockage : compression gzip pour logs archivés
- Conformité : norme ISO/IEC 27037 :2012 respectée

7.11 Module K : Explicabilité (XAI)

Équipe : 3 étudiants

Rôle : Expliquer les recommandations algorithmiques.

7.11.1 Fonctions principales (MVP)

- Justification textuelle simple pour chaque recommandation
- Affichage dans l'interface web (tooltip, popup)
- Traçabilité des raisonnements (pourquoi expert X? pourquoi incohérence Y?)

7.11.2 Extensions

- Visualisations (graphiques, heatmaps)
- Contrefactuels ("si on change X, le résultat devient Y")
- LIME/SHAP pour modèles ML complexes

7.11.3 Types d'explications

1. Matching experts (Module B) :

```
1 def explain_matching(expert: dict, score: float, breakdown: dict) -> str:
2     """Genere explication matching expert."""
3     return f"""
4 Expert {expert['nom']} selectionne avec score {score:.2f}:
5 - Similarite textuelle: {breakdown['text_score']:.2f}
6   (competences '{', '.join(expert['competences'])}' matchent
7   avec '{breakdown['matched_keywords']}' )
8 - Disponibilite: {breakdown['dispo_score']:.2f}
9 - Experience: {expert['annees_experience']} ans ({breakdown['exp_score']:.2
10 f})
11 """
```

2. Incohérences (Module F) :

```
1 def explain_incoherence(inc: dict) -> str:
2     """Explication incoherence detectee."""
3     if inc['type'] == 'temporelle':
4         return f"""
5 Incoherence temporelle detectee:
6 PV {inc['pv_references'][0]}: date {inc['date1']}
7 PV {inc['pv_references'][1]}: date {inc['date2']}
8 \textrightarrow Chronologie impossible (delta {inc['delta']} jours)
9 """
10     return "Explication generique"
```

3. Rapports scoring (Module H) :

```
1 def explain_rapport_score(rapport: dict, score: float) -> str:
2     """Explication scoring rapport."""
3     return f"""
4 Rapport '{rapport['action']}' note {score:.2f}:
5 - Completude: {score*0.4:.2f} (champs requis presents)
6 - Pertinence: {score*0.3:.2f} (mots-cles techniques)
7 - Preuves: {score*0.3:.2f} ({len(rapport['preuves'])} fichiers joints)
8 """
9 """
```

7.11.4 API exposée

GET /api/v1/xai/explain?module=B&entity_id=42


```

1 {
2   "module": "B",
3   "entity_id": 42,
4   "explanation": "Expert Alice Dubois selectionne avec score 0.87...",
5   "breakdown": {
6     "text_score": 0.72,
7     "dispo_score": 0.2,
8     "exp_score": 0.15
9   },
10  "visualizations": [
11    {
12      "type": "bar_chart",
13      "url": "/api/v1/xai/viz/chart-b-42.png"
14    }
15  ]
16 }

```

7.11.5 Dépendances

- **Consomme** : Modules B, F, H (résultats à expliquer)
- **Fournit à** : Module A (UI), Module I (rapport)

7.11.6 Livrables

- Générateurs d'explications (3 types minimum)
- Service FastAPI avec endpoint `/xai/explain`
- Composant React (tooltip explicatif)
- Tests unitaires (cohérence explications)

7.11.7 Critères d'acceptation

- Chaque recommandation accompagnée d'explication
- Explication compréhensible (test utilisateur)
- Affichage UI fonctionnel

7.11.8 Métriques de qualité

- Satisfaction utilisateur : $\geq 75\%$ (questionnaire)
- Temps génération : < 200 ms par explication
- Couverture : 100% des modules B, F, H

8 Tableau des interactions inter-modules

Module	Entrées (Inputs)	Sorties (Outputs)	Dépendances
A : Mandataire	Saisie mandataire (formulaire)	Mandat JSON, requêtes API	B, C, D
B : Matching	Mandat (A), base CV PostgreSQL	Top 3 profils JSON	A, D, K
C : Contrats	Mandat (A), expert (D)	PDF mandat + hash	A, D, J
D : Workflow	Profils (B), réponses experts	Expert choisi JSON	B, C, E
E : Préparation	Mandat (A), expert (D)	Questions, listes, guide PDF	A, D, F
F : PV Analyse	PV textes (externes), questions E	Entités NER, incohérences JSON	E, G, I, K
G : Hypothèses	Incohérences (F), type cas	Plan méthodologique JSON	F, H, I
H : Analyse Rapports	Rapports forensiques (G)	Chaîne optimale, scores JSON	G, I, K
I : Rapport Final	Résultats F, G, H	Rapport PDF complet + hash	F, G, H, J
J : Sécurité	Actions tous modules	Logs JSONL, hashes	Tous (A-I)
K : Explicabilité	Résultats B, F, H	Justifications textuelles	B, F, H, A

TABLE 1 – Tableau des interactions entre modules

9 Contraintes non-fonctionnelles

9.1 Performance

- Temps de réponse API : < 2 secondes (95^e percentile)
- Génération PDF : < 5 secondes
- NLP (module F) : < 3 secondes pour PV de 5000 mots
- Matching (module B) : < 1 seconde pour 100 experts

9.2 Sécurité

- HTTPS obligatoire (TLS 1.3)
- Authentification JWT (expiration 1h, refresh 7j)
- Hachage mots de passe : bcrypt (cost factor 12)
- Protection CSRF/XSS : headers sécurisés (Helmet.js)
- Validation des inputs : Pydantic côté backend, Zod côté frontend

9.3 Forensic et légal

- Conformité ISO/IEC 27037 :2012 (chaîne de custody)
- Hachage SHA-256 systématique
- Horodatage RFC 3161 (TSA simulé acceptable en MVP)
- Journaux immuables (append-only vérifié)
- RGPD : anonymisation des PV en option (pseudonymisation)

9.4 Scalabilité

- Architecture stateless (horizontal scaling possible)
- Pagination des résultats (max 50 items par page)
- Cache Redis pour requêtes fréquentes (optionnel)
- Base de données optimisée (index sur case_id, timestamps)

9.5 Disponibilité

- Uptime : $\geq 95\%$ (hors maintenance planifiée)
- Gestion des erreurs gracieuse (codes http standards)
- Retry automatique avec backoff exponentiel
- Health checks : /api/v1/health (toutes les 30s)

9.6 Maintenabilité

- Code coverage : $\geq 70\%$ par module
- Documentation API : OpenAPI 3.0 (Swagger UI)
- Logs structurés JSON (ELK compatible)
- Code quality : Black, Flake8, mypy (CI/CD)

10 Organisation et gouvernance

10.1 Répartition des équipes

Équipe	Module	Effectif
1	A : Interface Mandataire	3
2	B : Expert Matching	3
3	C : Générateur Contrats	3
4	D : Workflow Acceptation	3
5	E : Préparation Mission	3
6	F : Ingestion PV	3
7	G : Hypothèses	3
8	H : Analyse Rapports	3
9	I : Rapport Final	3
Intégration	J, K, API Gateway	9 (1 par équipe)

TABLE 2 – Répartition des équipes par module

10.2 Équipe d'intégration

Rôle : Assurer la cohérence inter-modules et gérer l'API Gateway centrale.

Composition : 1 membre de chaque équipe (9 étudiants).

Responsabilités :

- Définir les contrats d'interface (OpenAPI)
- Implémenter l'API Gateway (FastAPI + proxy)
- Coordonner les sprints (Scrum Master rotatif)
- Gérer le repository Git central (merge requests)
- Développer modules J et K (transverses)
- Organiser les tests d'intégration

10.3 Méthodologie Agile

- **Sprints :** 2 semaines (13 sprints sur 26 semaines)
- **Réunions :**
 - Daily stand-up équipe intégration (15 min, async Slack OK)
 - Sprint planning (2h, début de sprint)
 - Sprint review (1h30, fin de sprint)
 - Sprint retrospective (1h, fin de sprint)
- **Outils :**
 - Gestion projet : Jira ou GitHub Projects
 - Communication : Slack ou Discord
 - Documentation : Confluence ou Notion
 - Code review : GitHub Pull Requests (2 reviewers minimum)

10.4 Livrables individuels

Journal de bord : Minimum 20 entrées sur 26 semaines.

Contenu obligatoire :

- Date, durée travail
- Tâches réalisées (user stories)
- Difficultés rencontrées et solutions
- Apprentissages (techniques, méthodologiques)
- Participation sprints (planning, review, retro)
- Code reviews effectuées

Format : Markdown ou LaTeX, commit Git réguliers.

11 Planification détaillée

11.1 Semestre 1 (13 semaines) – MVP

11.1.1 Semaine 1 : Kickoff et spécifications

- Présentation projet et constitution équipes
- Lecture cahier des charges
- Formation stack technique (Python, FastAPI, React)
- Définition contrats d'interface (OpenAPI)

11.1.2 Sprints 1-2 (Semaines 2-5) : Modules A, B, C, D

Objectif : Flux création mandat → sélection expert → contrat.

User stories :

- A1 : En tant que mandataire, je peux créer un mandat
- B1 : Le système propose 3 experts pertinents
- D1 : Les experts peuvent accepter/refuser
- C1 : Le système génère un PDF de contrat

11.1.3 Sprints 3-4 (Semaines 6-9) : Modules E, J (partiel)

Objectif : Préparation mission + logs basiques.

User stories :

- E1 : Génération questions types
- J1 : Journalisation actions modules A-E
- J2 : Hachage fichiers PDF

11.1.4 Sprints 5-6 (Semaines 10-13) : Intégration MVP

Objectif : Test end-to-end + démo.

Livrables :

- Docker Compose fonctionnel (tous modules)
- Scénario démo : cas fraude bancaire complet
- Documentation API (Swagger)
- Présentation intermédiaire (20 min)

11.2 Semestre 2 (13 semaines) – Version complète

11.2.1 Sprints 7-9 (Semaines 14-19) : Modules F, G, H

Objectif : Analyse forensique complète.

User stories :

- F1 : Ingestion PV et extraction entités NER
- F2 : Détection incohérences (3 types)
- G1 : Génération hypothèses et plan méthodologique
- H1 : Scoring rapports techniques
- H2 : Chaîne optimale de résolution

11.2.2 Sprints 10-11 (Semaines 20-23) : Modules I, K

Objectif : Rapport final + explicabilité.

User stories :

- I1 : Compilation rapport PDF complet
- K1 : Explications matching experts
- K2 : Explications incohérences
- K3 : Explications scoring rapports

11.2.3 Sprints 12-13 (Semaines 24-26) : Finalisation

Objectif : Tests finaux, documentation, soutenance.

Tâches :

- Tests d'intégration complets (3 cas types)
- Corrections bugs critiques
- Rédaction rapport final (30-40 pages)
- Préparation soutenance + vidéo démo
- Finalisation journaux de bord individuels

11.3 Diagramme de Gantt (simplifié)

Module	S1-5	S6-9	S10-13	S14-19	S20-23	S24-26
A, B, C, D	XXX	.	.	.	.	.
E, J (part)	.	XXX	.	.	.	.
Intégration MVP	.	.	XXX	.	.	.
F, G, H	.	.	.	XXX	.	.
I, K	.	.	.	.	XXX	.
Finalisation	.	.	.	.	.	XXX

TABLE 3 – Diagramme de Gantt simplifié du projet

12 Gestion des risques

12.1 Matrice de risques

Risque	Impact	Proba	Mitigation	Responsable
Désynchronisation équipes	Élevé	Élevé	Stand-up quotidien équipe intégration, contrats d'interface stricts	Équipe intégration
Module F (NLP) trop complexe	Moyen	Moyen	Fallback : extraction manuelle guidée, spaCy pré-entraîné	Équipe F + Prof
Absence membre clé	Moyen	Faible	Documentation exhaustive, pair programming, bus factor ≥ 2	Chef d'équipe
Dérive planning	Élevé	Moyen	Buffer 1 semaine semestre 2, priorisation MVP strict	Équipe intégration
Bugs intégration	Élevé	Élevé	CI/CD dès sprint 2, tests automatisés, code reviews	Équipe intégration
Stack technique inadap-tée	Faible	Faible	POC semaine 1, formation obligatoire	Prof + Équipe intégration
Complexité légale (RGPD)	Faible	Moyen	Anonymisation optionnelle MVP, expert légal consulté	Prof

TABLE 4 – Matrice des risques du projet

12.2 Plan de contingence

Si retard > 2 semaines :

1. Réduire périmètre : retirer modules G, H (extensions)
2. Focus MVP strict : A, B, C, D, E, F, I, J
3. Sprint supplémentaire semestre 2 (réduire vacances)

Si module critique bloqué :

1. Mock API temporaire (données fixtures)
2. Réaffectation ressources (aide autre équipe)
3. Escalade au professeur (session débogage)

13 Stratégie de test

13.1 Tests unitaires

- **Framework** : pytest (Python), Jest (TypeScript)
- **Coverage** : $\geq 70\%$ par module
- **Exécution** : CI/CD à chaque push
- **Exemples** :

```
1 def test_expert_matching():
2     mandat = {"case_type": "fraude", "description": "..."}
3     experts = match_experts(mandat, mock_db)
4     assert len(experts) == 3
5     assert experts[0]['score'] > experts[1]['score']
6
7 def test_pdf_generation():
8     contract = generate_contract_pdf(mock_data)
9     assert os.path.exists(contract['pdf_path'])
10    assert len(contract['hash']) == 64 # SHA-256
11
```

13.2 Tests d'intégration

- **Framework** : pytest avec fixtures, requests
- **Scénarios obligatoires** :
 1. Cas fraude bancaire (flux complet A→I)
 2. Cas intrusion réseau (flux complet)
 3. Cas malware (flux complet)
- **Exemple** :

```
1 def test_end_to_end_fraude():
2     # 1. Creer mandat
3     r = requests.post(API_URL + "/cases", json=mandat_fraude)
4     case_id = r.json()['case_id']
5
6     # 2. Obtenir experts
7     r = requests.get(API_URL + f"/cases/{case_id}/experts")
8     experts = r.json()['experts']
9     assert len(experts) == 3
10
11    # 3. Accepter expert
12    r = requests.put(API_URL + f"/solicitations/{experts[0]['id']}/respond",
```

```

13         json={"status": "accepted"})
14
15     # 4. Generer contrat
16     r = requests.post(API_URL + "/contracts/generate", json={...})
17     assert r.status_code == 201
18
19     # ... suite du flux

```

13.3 Tests end-to-end (E2E)

- **Framework** : Cypress (frontend)
- **Scénarios** :
 - Mandataire crée cas → visualise experts → télécharge PDF
 - Expert accepte mission → reçoit guide

13.4 Tests de performance

- **Outil** : Locust ou Apache Bench
- **Objectifs** :
 - 100 requêtes/s (API Gateway)
 - Temps réponse < 2s (95^e percentile)

14 Critères d'évaluation

14.1 Grille d'évaluation globale

Critère	Poids	Description
Cahier des charges	10%	Complétude, clarté, spécifications techniques
Implémentation MVP	35%	Fonctionnalités A-E + J, qualité code, tests
Rapport final + soutenance	30%	Rédaction, démo, réponses questions
Journal de bord individuel	15%	Régularité, qualité réflexion, participation
Déontologie / sécurité	10%	Respect ISO 27037, RGPD, chain of custody

TABLE 5 – Grille d'évaluation globale

14.2 Critères détaillés par module

Module	Fonctionnel	Technique	Qualité code
A	UI fonctionnelle	React + TypeScript	Tests E2E
B	Top 3 pertinents	TF-IDF + PostgreSQL	Coverage $\geq 80\%$
C	PDF valide	LaTeX + Jinja2	Gestion erreurs
D	Workflow complet	Machine à états	Tests unitaires
E	Questions générées	Règles + templates	Extensibilité
F	NER $\geq 80\%$	spaCy + NLP	Dataset test
G	Plan cohérent	Base connaissance	Validation expert
H	Scoring correct	Algorithme scoring	Tests unitaires
I	Rapport complet	LaTeX compilation	PDF professionnel
J	Logs 100%	Append-only + hash	ISO 27037
K	Explications claires	Générateurs texte	Satisfaction user

TABLE 6 – Critères d’évaluation par module

14.3 Bonus (extensions)

- Module G, H implémentés : +5%
- Signatures électroniques qualifiées : +3%
- Dashboard temps réel (WebSocket) : +2%
- Deep learning (module F) : +3%
- Deployment production (Heroku, AWS) : +2%

15 Annexes

15.1 Annexe A : Templates obligatoires

15.1.1 Template mandat (LaTeX)

Voir module C pour template complet.

15.1.2 Template PV (texte structuré)

PROCES-VERBAL D'AUDITION

Date: [DATE]

Cas: [CASE_ID]

Temoin/Suspect: [NOM]

Investigateur: [NOM_EXPERT]

DECLARATION:

[Texte libre de la declaration, 500-5000 mots]

QUESTIONS POSEES:

1. [Question 1]

Reponse: [...]

2. [Question 2]

Reponse: [...]

Signature témoin: -----
Signature investigateur: -----

15.1.3 Template rapport final (LaTeX)

Voir module I pour structure complète.

15.2 Annexe B : Dataset de test

15.2.1 Cas 1 : Fraude bancaire

Description : Virements suspects de 50 000€ vers compte étranger.
PV témoin 1 : "J'ai autorisé le virement le 10/10/2025 à 14h."
PV témoin 2 : "Le compte a été compromis le 05/10/2025."
Incohérence attendue : Temporelle (virement après compromission impossible).

15.2.2 Cas 2 : Intrusion réseau

Description : Détection connexions suspectes depuis IP chinoise.
Logs : Connexion SSH 192.168.1.50 à 03 :42, exfiltration 2 GB.
Incohérence attendue : Géographique (employé en vacances à Paris).

15.2.3 Cas 3 : Malware

Description : Ransomware chiffre serveur fichiers.
PV : "J'ai cliqué sur la pièce jointe mardi matin."
Incohérence attendue : Factuelle (email reçu mercredi selon logs).

15.3 Annexe C : Contraintes RGPD et ISO 27037

15.3.1 RGPD (Règlement Général Protection Données)

- **Pseudonymisation :** Remplacer noms réels par identifiants (M. A, Mme B)
- **Minimisation :** Collecter uniquement données nécessaires
- **Durée conservation :** Supprimer données après fin mission (+6 mois)
- **Droit accès :** Permettre téléchargement données personnelles
- **Sécurité :** Chiffrement au repos (AES-256) et en transit (TLS)

Mise en œuvre MVP :

- Pseudonymisation optionnelle (flag dans API)
- Chiffrement TLS obligatoire
- Mention RGPD dans rapport final

15.3.2 ISO/IEC 27037 :2012 (Digital Evidence)

- **Identification :** Documenter source preuve (PV, log, fichier)
- **Collection :** Hacher SHA-256 immédiatement
- **Acquisition :** Copie bit-à-bit (hors périmètre MVP)
- **Préservation :** Logs immuables, stockage sécurisé
- **Documentation :** Qui, quoi, quand, où, comment

Mise en œuvre MVP :

- Module J : hachage + horodatage systématique

- Logs structurés avec 5W1H (who, what, when, where, why, how)
- Rapport final : section "Chaîne de custody"

15.4 Annexe D : Spécifications API Gateway

15.4.1 Rôle

- Point d'entrée unique pour tous les modules
- Routage des requêtes (proxy inverse)
- Authentification centralisée (JWT)
- Rate limiting (100 req/min par IP)
- Logs centralisés

15.4.2 Architecture

```

1 # app/gateway.py
2 from fastapi import FastAPI, Request
3 import httpx
4
5 app = FastAPI(title="API Gateway")
6
7 # Mapping modules
8 MODULE_ROUTES = {
9     '/api/v1/cases': 'http://module-a:8000',
10    '/api/v1/matching': 'http://module-b:8001',
11    '/api/v1/contracts': 'http://module-c:8002'
12 }
13
14 @app.api_route("/{path:path}", methods=["GET", "POST", "PUT", "DELETE"])
15 async def gateway(request: Request, path: str):
16     """Proxy toutes les requetes vers les modules."""
17     # 1. Authentification JWT
18     verify_jwt(request.headers.get('Authorization'))
19
20     # 2. Determiner module cible
21     target_url = None
22     for prefix, base_url in MODULE_ROUTES.items():
23         if request.url.path.startswith(prefix):
24             target_url = base_url + request.url.path
25             break
26
27     if not target_url:
28         return {"error": "Route not found"}, 404
29
30     # 3. Forwarder requete
31     async with httpx.AsyncClient() as client:
32         response = await client.request(
33             method=request.method,
34             url=target_url,
35             headers=request.headers,
36             content=await request.body()
37         )
38
39     # 4. Logger action (module J)
40     log_action(request.method, request.url.path, response.status_code)
41
42     return response.json(), response.status_code

```

15.4.3 Déploiement

```
1 # docker-compose.yml
2 version: '3.8'
3 services:
4   gateway:
5     build: ./gateway
6     ports:
7       - "8000:8000"
8     environment:
9       - JWT_SECRET=...
10    depends_on:
11      - module-a
12      - module-b
13
14  module-a:
15    build: ./modules/a
16    ports:
17      - "8001:8000"
18
19  # ... autres modules
```

15.5 Annexe E : Glossaire

Mandataire Personne émettant le mandat (juge, chef entreprise, particulier)

Expert Investigateur numérique désigné pour la mission

PV Procès-Verbal d'audition (témoin ou suspect)

NER Named Entity Recognition (extraction entités)

XAI Explainable AI (intelligence artificielle explicable)

Chain of custody Chaîne de custody forensique (traçabilité preuves)

MVP Minimum Viable Product (produit minimal viable)

TF-IDF Term Frequency-Inverse Document Frequency (pondération termes)

JWT JSON Web Token (authentification)

CI/CD Continuous Integration/Continuous Deployment (intégration continue)

15.6 Annexe F : Références

1. ISO/IEC 27037 :2012 – Guidelines for identification, collection, acquisition and preservation of digital evidence
2. RGPD – Règlement (UE) 2016/679 du Parlement européen
3. Casey, E. (2011). *Digital Evidence and Computer Crime*. Academic Press.
4. NIST SP 800-86 – Guide to Integrating Forensic Techniques into Incident Response
5. FastAPI Documentation : <https://fastapi.tiangolo.com>
6. spaCy Documentation : <https://spacy.io>
7. RFC 3161 – Time-Stamp Protocol (TSP)